

# First SAFS Run — Workflow Map

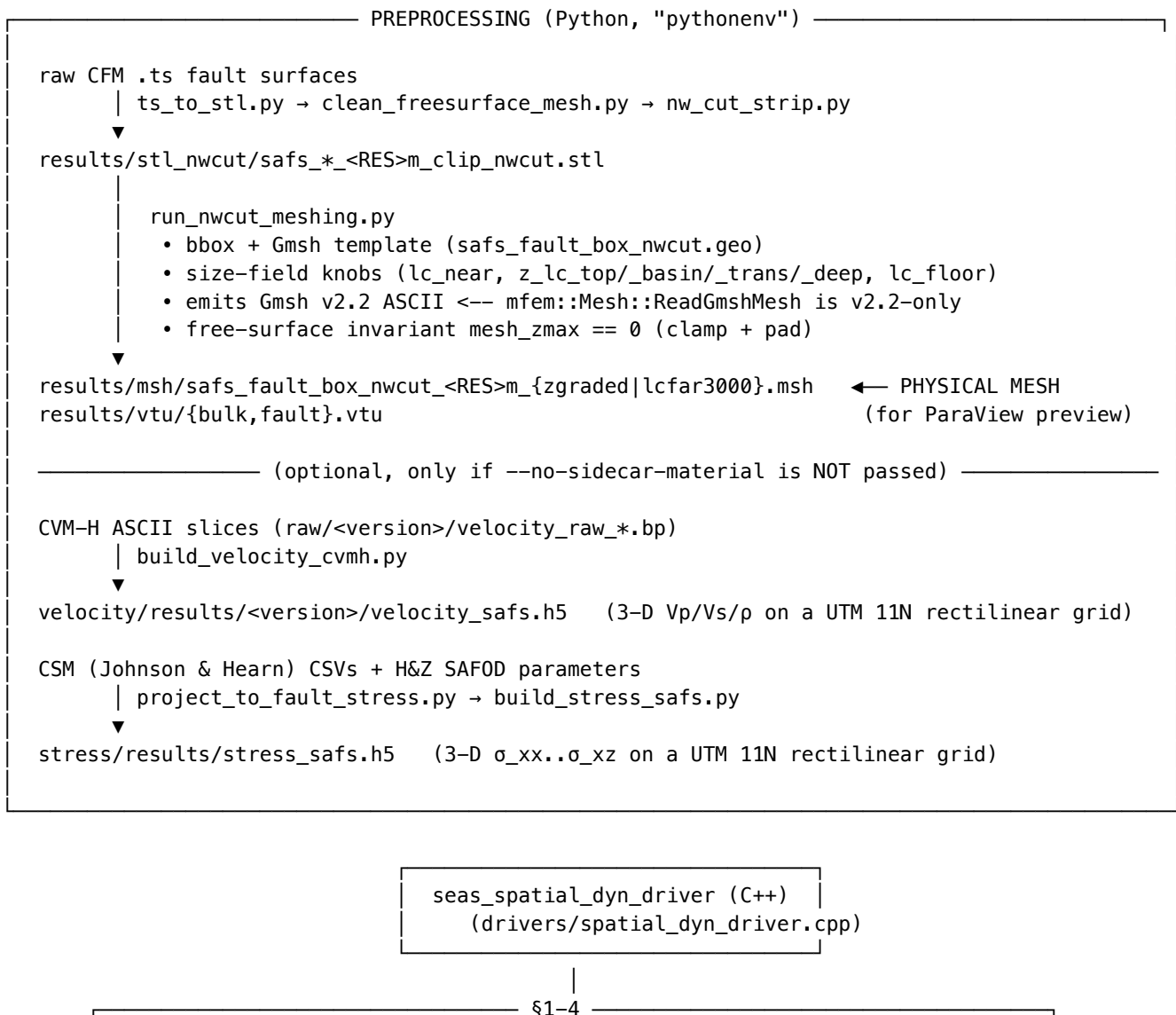
Reference plan: safs/project\_7.0\_alternative/document/05\_18\_2026/PLAN\_first\_safs\_run.md Driver: miniapps/seas/drivers/spatial\_dyn\_driver.cpp binary seas\_spatial\_dyn\_driverTOMLexample: safs/project\_7.0\_alt Mesh family: safs/project\_7.0\_alternative/meshing/results/msh/safs\_fault\_box\_nwcut\_\*m\_{zgraded,lcfar3000}.msh

## Overview

A first SAFS production run executes a single-event dynamic rupture on the SAF ALT6 curvilinear fault embedded in a 3-D bulk half-space. Three Python preprocessing pipelines (meshing/, velocity/, stress/) produce the mesh + optional CVM material sidecar + CSM bulk-stress sidecar. One C++ driver (seas\_spatial\_dyn\_driver) loads them, builds per-DOF fault tables, projects the stress onto the fault, resolves LSW friction + a gradual overstress nucleation, and integrates the velocity-stress wave system with ADER + TPV205-style LSW sub-step iteration. Output is a single fault.vtkhdf + optional volume velocity.vtkhdf and bulk stress.vtkhdf, with a per-rank cp\_\* V1 restart checkpoint every N steps.

The byte-exact regression contract for native TPV\* and BP5 drivers stays green across every commit of this work — Phase N's destructive removals (the old Overstress / StrengthReduction enums + LSW\_ForcedRupture dispatch) touch only the spatial code path.

## Workflow Map (plain text)



```

CLI parse (--config TOML; CLI overrides win)
TOML load → SpatialFrictionConfig (mesh path, [stress], [friction.*],
                                   [nucleation.gradual_overstress], ...)
load mesh + ParMesh + SetCurvature(order)

```

#### §5-7

```

BoundaryConfig: fault=101, top free=102, sides absorbing=103+104
MaterialField
  └ Constant ⇒ from [material_constant_fallback] (--no-sidecar-material)
  └ Coefficient ⇒ ABORT (Phase H gap – heterogeneous ctor not yet wired)
WaveOperator<ParMesh>(pmesh, order, λ, μ, ρ, bc) ← scalar-material ctor
wave.SetFaultFrictionLaw(LSW) // Phase N: never LSW_ForcedRupture in SAFS

```

#### §8-9

```

Build per-DOF fault tables (inline; spatial::ComputePerDOFCoords... deferred)
  fault_int_faces, fault_shr_faces <-- wave.GetFault{Interior,Shared}Faces()
  nbf_per_face <-- ProbeNbfPerFace + MPI_Allreduce(MAX)
  FaultBasis.Compute / AppendSharedFaces (ref_normal=+y, up=+z; BP5 conv.)
  BuildPerDOFFaultTables → dof_coords_3d, dof_basis(9,N), dof_to_elem,
                        dof_to_attr, dof_ips
  dof_basis layout per DOF i:  rows 0..2 = normal,
                               rows 3..5 = tangent1 = DIP,
                               rows 6..8 = tangent2 = STRIKE
  FaultGeometry<ParMesh> geom(bp5_seed, ..., &mpi_ctx, dof_ips)

```

#### §10

```

Stress projection onto fault tractions:
  kind == "constant_tensor" ⇒ ConstantTensorStressSource
                             + geom.ComputeSAFSParams<StressSource>()
  kind == "sidecar_hdf5"    ⇒ ApplyCsmStressSidecar(spec, geom)
                             internally: load stress_safs.h5 →
                             StressField3D → geom.ComputeSAFSParams
After: geom.GetTauPre() (2*N: [τ_dip, τ_strike] per DOF, τ || V_init)
      geom.sigma_n_per_dof() (N: σ_n > 0 compression, P_p subtracted)

```

#### §11-12

```

SpatialFrictionResolver::ResolveSlipWeakening
  walks [[friction.slip_weakening.spatial]] rules (in document order,
  last-match wins per key); produces SlipWeakeningPerDOFParams
  {mu_s(i), mu_d(i), d_c(i), cohesion(i)} for every fault DOF.
  "barrier" rule sets mu_s = 1.0e6 sentinel (skipped by --print-derived).

spatial::ResolveGradualOverstress (Phase N):
  For each DOF i:
    F(r_i) = GaussianFactorFaceLocal(coords, dof_basis[dip,strike],
                                     center, radius_dip_m, radius_strike_m)
    amplitude_dip(i)   = F(r_i) · Δτ_dip
    amplitude_strike(i) = F(r_i) · Δτ_strike
    radial(i)          = F(r_i)
  When [nucleation] block absent ⇒ three zero-sized Vectors (no-op later).

```

#### §13-14

```

FaultFaceFlux fault_flux(p, cp_seed, cs_seed)
wave.SetFaultFlux(&fault_flux)
spatial::InitializeFaultDOFs_Spatial<ParMesh>(
    dof_data, num_fault_total, dof_to_elem, material, pmesh, lsw,
    geom.GetTauPre(), geom.sigma_n_per_dof(),
    dummy_T_forced(=1e9), dummy_t0_decay(=0)    // sentinels, inert in LSW
    dof_ips)
wave.SetFaultDOFData(&dof_data, nbf_per_face)
Q_bg = 0 absorbing background

```

§15

```

dt_cfl = wave.ComputeMaxDt(cfl)    // dt = cfl · h_min / cp (scalar)
dt = min(dt_cfl, dt_max)
nsteps = ceil(tfinal / dt)

```

§16 – pre-flight + dry-run gate

```

--print-derived ⇒ spatial::PrintDerivedAndCheck (Phase D)
    L_nuc histogram, |τ_pre|/(μ_s σ_n_eff) outside-asperity gate,
    nucleation budget gate, σ_1 azimuth/plunge, CFL summary,
    NumZeroNormalFallbacks; ABORT on ill-posed setup.
--dry-run ⇒ MPI_Finalize and exit 0 without entering the time loop.

```

§17

```

ParaView output wiring (Parity Phases 1–6). Master gate paraview_enabled:
    primary pv_out → "<output_dir>/volume.vtkhdf" + fault.vtkhdf
        registers: velocity (3-comp L2), mpi_rank (L2 p=0),
            8 SAFS fault static fields via SetFaultParamsSpatial:
                lsw_mu_s, lsw_mu_d, lsw_d_c, nuc_amplitude,
                nuc_radial_factor, sigma_n_init, tau1_init, tau2_init
    secondary pv_bulk_out → "<output_dir>/ParaView_bulk/stress.vtkhdf"
        registers: sigma_xx, sigma_yy, sigma_zz, sigma_xy, sigma_yz, sigma_xz
All HDF5 collections optionally ZFP-compressed (bulk/volume ~1e-3,
fault 1e-12 to preserve the slip-rate dynamic range).

```

§18

```

Restart from --restart <prefix> ⇒ ReadTpv104Checkpoint
    reads (t, dt, step, Q, dof_data) from per-rank cp_t*.h5 files.
    Refuses if driver_tag != "spatial_dyn".
Output-dir == restart-dir ⇒ exit 3 (overwrite-guard).

```

§19

```

Tpv205SubStepIterator substep_iterator(fault_flux)
    SetSubSteps(deltaT[0], weights[0])    // 0 = ader_order

Phase N nucleation callback (closes over nuc_params + dof_data + cfg):
    nuc_cb = λ(t_sub_end, dt_sub):
        if cfg.nucleation.enabled:
            spatial::ApplyGradualOverstressIncrement(
                dof_data, nuc_params, T_nuc_s, t_sub_end, dt_sub);
            ΔS = SmoothStepIncrement(t_sub_end, dt_sub, T_nuc_s)
            per DOF i: tau1_nuc[i] += ΔS · amplitude_dip(i)
                       tau2_nuc[i] += ΔS · amplitude_strike(i)

```

```

|
|
|----- $20 (time loop) -----
for step in [step0, nsteps):
    dt_step = min(dt_now, tfinal - t)
    AdvanceADERWithSubStep_Spatial(wave, iterator, dof_data, ...,
                                    Q, dt_step, ader_order, t, Q_new, nuc_cb)
    1. Compute 0 sub-step bulk states Q_per_node via
        wave.ComputeADERSubStepStates (ADER predictor in time)
    2. EvaluateBulkAtFaultQPsCanonical → Q_pointwise_{plus,minus} per node
    3. iterator.AdvanceWithSubStepStates(dof_data, fault_coords, Q±, ...,
                                        I_imp_{plus,minus}, nuc_cb)

        for o in [0, 0):
            nuc_cb(t_sub_end, dt_sub) ← Phase N injection
            StepOneQP(d, Q+, Q-, dof_data[i]) → solves LSW closed-form
            Brent for |V|, updates slip, V, tau_corr, sigma_n_corr,
            writes I_imp side-channel for the corrector
    4. wave.SetSubStepFaultImposedStates(I_imp±)
    5. wave.AdvanceADER(Q, dt_step, ader_order, Q_new) (corrector)
    Q.Swap(Q_new); t += dt_step
    V_max_step = MPI_Allreduce(MAX, local V_max over dof_data)
    paraview_write(step+1, t, V_max_step)
    every checkpoint_every_steps ⇒ WriteTpv104Checkpoint(..., "spatial_dyn")

```

```

|
|----- $21 + Phase Z -----
Final WriteTpv104Checkpoint, MPI_Finalize.
Postprocess: scripts/verify_spatial_dyn_smoke_safs.py
    opens fault.vtkhdf, asserts no NaN,  $\sigma_n > 0$ , V_max unimodal,
    rupture-core V_max  $\geq 1e-3$  m/s after T_nuc_s; writes verify_summary.txt.

```

## Call Graph (critical path)

```

main()
├─ LoadSpatialFrictionConfig(toml) // spatial_dyn_driver.cpp:447
├─ ParMesh(comm, Mesh(cfg.mesh.path)) // spatial_friction.cpp
├─ MaterialField::MakeConstant( $\lambda, \mu, \rho$ )
├─ WaveOperator<ParMesh>(pmesh, order,  $\lambda, \mu, \rho, bc$ )
├─ wave.SetFaultFrictionLaw(LSW) // Phase N: always LSW
├─ BuildPerDOFFaultTables(...) → dof_coords_3d, dof_basis(9,N)
├─ FaultGeometry<ParMesh>(bp5_seed, ..., &mpi_ctx, dof_ips)
├─ geom.ComputeSAFSParams(ConstantTensorStressSource | sidecar)
├─   └─ FieldProjector::ProjectFaultPreStress
├─ SpatialFrictionResolver::ResolveSlipWeakening(cfg, coords, attr)
├─ spatial::ResolveGradualOverstress(spec, enabled, coords, basis)
├─   └─ for each DOF: GaussianFactorFaceLocal(...)
├─ FaultFaceFlux fault_flux( $\rho, cp\_seed, cs\_seed$ )
├─ spatial::InitializeFaultDOFs_Spatial(dof_data, ..., lsw,  $\tau\_pre, \sigma\_n\_eff, dof\_ips$ )
├─ dt_cfl = wave.ComputeMaxDt(cfl)
├─ if --print-derived: spatial::PrintDerivedAndCheck(...) // Phase D
├─ if --dry-run: MPI_Finalize; return 0
├─ ParaViewOutput<ParMesh> pv_out + pv_bulk_out // Parity Phases 1-6
├─   └─ RegisterDomainField("velocity"|"mpi_rank"|"sigma_*")
├─   └─ InitFaultOutputBP5(fault_int_faces, fault_shr_faces, nbf)
├─   └─ SetFaultParamsSpatial({8 static SAFS fields})

```

```

- if --restart: ReadTpv104Checkpoint(prefix, t, dt, step, Q, dof_data, ...)
- Tpv205SubStepIterator iterator(fault_flux); SetSubSteps(0)
- nuc_cb =  $\lambda(t_{sub\_end}, dt_{sub})$ : // Phase N hook
-   spatial::ApplyGradualOverstressIncrement(dof_data, nuc_params, T_nuc_s, ...)
- for step in [step0, nsteps):
-   AdvanceADERWithSubStep_Spatial(wave, iterator, dof_data, fault_coords,
-                                   Q, dt_step, ader_order, t, Q_new, nuc_cb)
-   wave.ComputeADERSubStepStates(Q, dt, 0, tau_nodes, Q_per_node)
-   wave.EvaluateBulkAtFaultQPsCanonical(Q_per_node[0], Q $\pm$ [0]) // for each o
-   iterator.AdvanceWithSubStepStates(dof_data, fault_coords,
-                                     Q+, Q-, dt, t, I_imp $\pm$ , nuc_cb)
-   for o in [0, 0):
-     nuc_cb(t_sub_end, dt_sub) // Phase N
-     for each fault DOF i:
-       StepOneQP_(dof_data[i], Q+, Q-, ...) // LSW Brent
-   wave.SetSubStepFaultImposedStates(I_imp $\pm$ )
-   wave.AdvanceADER(Q, dt, 0, Q_new) // corrector
-   paraview_write(step+1, t, V_max_step) // adaptive
-   WriteTpv104Checkpoint(prefix, t, dt, step+1, Q, dof_data, "spatial_dyn")

```

---

## Key code snippets

### A. Driver header — what gets included

```

// drivers/spatial_dyn_driver.cpp:47-80
#include "mfem.hpp"

#include "../dynamic/wave_state.hpp"
#include "../dynamic/wave_operator.hpp"
#include "../dynamic/fault_face_flux.hpp"
#include "../dynamic/friction_solver.hpp"
#include "../dynamic/tpv205_friction.hpp"
#include "../dynamic/tpv205_substep_iterator.hpp"
#include "../dynamic/heterogeneous_material.hpp"
#include "../dynamic/spatial_setup.hpp"
#include "../dynamic/seas_diag_rank.hpp"

#include "../domain/boundary_config.hpp"

#include "../fault/fault_basis.hpp"
#include "../fault/fault_geometry.hpp"
#include "../fault/fault_geometry_safs_templated.inl"

#include "../config/bp5_params.hpp"

#include "../io/paraview_output.hpp"
#include "../io/tpv104_checkpoint.hpp"
#include "../io/data_field_3d.hpp"
#include "../io/stress_field_3d.hpp"

#include "../dynamic/spatial_nucleation.hpp" // Phase N
#include "../dynamic/spatial_print_derived.hpp" // Phase D

#include "../spatial/code/spatial_friction.hpp"
#include "../spatial/code/spatial_velocity.hpp"

```

```
#include "../spatial/code/spatial_stress.hpp"
```

Reading top-to-bottom: bulk velocity-stress wave (wave\_state, wave\_operator), fault constitutive machinery (fault\_face\_flux, friction\_solver, tpv205\_friction, tpv205\_substep\_iterator), material handling (heterogeneous\_material, spatial\_setup), fault geometry (fault\_basis, fault\_geometry, fault\_geometry\_safs\_templated.inl), output + restart (paraview\_output, tpv104\_checkpoint), and the SAFS-only glue (spatial\_nucleation, spatial\_print\_derived, the spatial/code/\* config/velocity/stress parsers).

## B. Per-DOF fault-table walk

The driver does not yet use the deferred ComputePerDOFCoordsAndBasisFromWave helper; instead it inlines the walk (driver lines 239-331). The crucial fact is the dof\_basis layout — column-major (9, N) with rows [normal; tangent1=dip; tangent2=strike]:

```
// drivers/spatial_dyn_driver.cpp:265-285
auto write_dof = [&](int dof_idx, FaceElementTransformations *ftr,
                    const IntegrationPoint &ip,
                    const FaultBasisData &bdata,
                    int attr)
{
    ftr->SetAllIntPoints(&ip);
    Vector phys(3);
    ftr->Face->Transform(ip, phys);
    phys_coords.push_back(phys);

    for (int d = 0; d < 3; ++d)
    {
        dof_coords_3d(3 * dof_idx + d) = phys(d);
        dof_basis(0 + d, dof_idx) = bdata.normal[d]; // rows 0..2 = n
        dof_basis(3 + d, dof_idx) = bdata.tangent1[d]; // rows 3..5 = dip
        dof_basis(6 + d, dof_idx) = bdata.tangent2[d]; // rows 6..8 = strike
    }
    dof_to_elem[dof_idx] = ftr->Elem1No;
    dof_to_attr[dof_idx] = attr;
    dof_ips[dof_idx] = ftr->GetElement1IntPoint();
};
```

Interior faces are walked first, shared faces appended — so the per-rank [0, num\_fault\_local) range is interior-fault DOFs and [num\_fault\_local, num\_fault\_total) is shared-fault DOFs.

## C. Stress projection (constant tensor branch)

```
// drivers/spatial_dyn_driver.cpp:1004-1020
if (cfg.stress.kind == spatial::StressSourceKind::ConstantTensor)
{
    spatial::ConstantTensorStressSource src(cfg.stress.sigma_xx_pa,
                                             cfg.stress.sigma_yy_pa,
                                             cfg.stress.sigma_zz_pa,
                                             cfg.stress.sigma_xy_pa,
                                             cfg.stress.sigma_yz_pa,
                                             cfg.stress.sigma_xz_pa);

    geom.ComputeSAFSParams(src,
                           cfg.stress.pore_pressure.P_p_pa,
                           cfg.stress.pore_pressure.P_p_grad_pa_per_m,
                           cfg.stress.pore_pressure.min_sigma_n_pa);
}
else
{

```

```

    spatial::ApplyCsmStressSidecar(cfg.stress, geom);
}

```

The templated `FaultGeometry<ParMesh>::ComputeSAFSParams<StressSource>` (defined in `fault/fault_geometry_safs_template.h`) walks every fault DOF, evaluates the supplied stress source at the DOF's UTM coords, resolves the 3×3 Cauchy tensor onto the local fault basis to get  $(\tau_{\text{dip}}, \tau_{\text{strike}}, \sigma_{\text{n\_total}})$ , subtracts pore pressure, clamps with `min_sigma_n_pa`, and stores into `geom.tau_pre_` (size  $2 * N$ , interleaved [dip, strike]) and `geom.sigma_n_per_dof_` (size  $N$ ).

The SCEC  $\tau_{\text{pre}} \parallel V_{\text{init}}$  invariant (CLAUDE.md §“Sign Conventions”) is maintained because `ConstantTensorStressSource` rotates the bulk tensor through the same `FaultBasisData` (dip, strike, normal) that `InitializeFaultDOFsSpatial` later consumes for  $V_{\text{init}}$ .

#### D. Phase N — gradual overstress resolve and accumulate

```

// drivers/spatial_dyn_driver.cpp:1043-1048 (resolve, ONCE at init)
const spatial::GradualOverstressPerDOFParams nuc_params =
    spatial::ResolveGradualOverstress(
        cfg.nucleation.gradual_overstress,
        cfg.nucleation.enabled,
        dof_coords_3d,
        dof_basis);

```

The accumulator runs once per ADER sub-step inside the iterator hook:

```

// drivers/spatial_dyn_driver.cpp:1580-1588 (per-sub-step callback)
auto nuc_cb = [&nuc_params, &dof_data, &cfg]
    (real_t t_sub_end, real_t dt_sub)
{
    if (!cfg.nucleation.enabled) { return; }
    spatial::ApplyGradualOverstressIncrement(
        dof_data, nuc_params,
        cfg.nucleation.gradual_overstress.T_nuc_s,
        t_sub_end, dt_sub);
};

```

Mathematically, for each fault DOF  $i$ , every sub-step adds  $\Delta S \cdot F(r_i) \cdot \Delta \tau$  to `DOFData[i].tau{1,2}_nuc`, where  $\Delta S = \text{smoothStep}(t_{\text{sub\_end}}, T_{\text{nuc\_s}}) - \text{smoothStep}(t_{\text{sub\_end}} - dt_{\text{sub}}, T_{\text{nuc\_s}})$  telescopes exactly to 1.0 over  $[0, T_{\text{nuc\_s}}]$ . The LSW solver consumes  $s.\text{tau}\{1,2\}_{\text{total}} = \text{tau}\{1,2\}_0 + \text{tau}\{1,2\}_{\text{nuc}} + \text{trial}$  per QP, so the nucleation perturbation is visible to Brent's friction-law root finder.

#### E. Time-loop body — ADER + sub-step LSW

```

// drivers/spatial_dyn_driver.cpp:1680-1727
for (int step = step0; step < nsteps; ++step)
{
    const real_t dt_step = std::min(dt_now, cfg.time.tfinal - t);
    if (dt_step <= 0.0) { break; }
    wave.SetTime(t);

    AdvanceADERWithSubStep_Spatial(wave, substep_iterator, dof_data,
                                    fault_coords, Q, dt_step,
                                    cfg.numerics.ader_order, t, Q_new,
                                    nuc_cb);

    Q.Swap(Q_new);
    t += dt_step;
    last_completed_step = step + 1;

    real_t V_max_local = 0.0;
}

```

```

for (int i = 0; i < num_fault_total; ++i)
{ V_max_local = std::max(V_max_local, dof_data[i].slip_rate); }
real_t V_max_step = V_max_local;
MPI_Allreduce(&V_max_local, &V_max_step, 1,
              MPITypeMap<real_t>::mpi_type, MPI_MAX, comm);
V_max_global = std::max(V_max_global, V_max_step);

paraview_write(step + 1, t, V_max_step);

if (cfg.output.checkpoint_every_steps > 0
    && (step + 1) % cfg.output.checkpoint_every_steps == 0)
{
    WriteTpv104Checkpoint(prefix, t, dt_now, step + 1, Q, dof_data,
                          rank, nprocs, comm, "spatial_dyn");
}
}

```

AdvanceADERWithSubStep\_Spatial (driver lines 343-439) is the SAFS wrapper:

```

// drivers/spatial_dyn_driver.cpp:394-424 (heart of the macro-step)
std::vector<Vector> Q_per_node;
wave.ComputeADERSubStepStates(Q, dt_step, ader_order, tau_nodes, Q_per_node);

std::vector<std::vector<real_t>> Q_pointwise_plus(0), Q_pointwise_minus(0);
for (int o = 0; o < 0; ++o)
{
    wave.EvaluateBulkAtFaultQPsCanonical(Q_per_node[o],
                                         Q_pointwise_plus[o],
                                         Q_pointwise_minus[o]);
}

std::vector<real_t> I_imp_plus_flat(n_words, 0.0);
std::vector<real_t> I_imp_minus_flat(n_words, 0.0);

if (n_total_fault_qps > 0)
{
    iterator.AdvanceWithSubStepStates(dof_data, fault_coords,
                                       Q_pointwise_plus,
                                       Q_pointwise_minus,
                                       dt_step, t_step_start,
                                       I_imp_plus_flat.data(),
                                       I_imp_minus_flat.data(),
                                       nuc_callback); // Phase N
}

wave.SetSubStepFaultImposedStates(...);
wave.AdvanceADER(Q, dt_step, ader_order, Q_new); // corrector

```

The pattern is: 1. Predictor — ComputeADERSubStepStates builds an order-0 polynomial of  $Q(t + \tau)$  and samples it at the 0 sub-step midpoints  $\tau_o$ . 2. Trace to fault — EvaluateBulkAtFaultQPsCanonical evaluates the bulk state on both sides (+ and -) of every fault QP. 3. Sub-step LSW — the Tpv205SubStepIterator walks the 0 sub-steps; for each it (a) calls nuc\_cb to inject the gradual-overstress increment, then (b) calls StepOneQP\_ per fault QP to solve the closed-form LSW Brent root, advance slip / slip-rate, and accumulate the imposed-flux side-channel  $I_{\text{imp}\pm}$  (which carries the friction-corrected traction back to the bulk). 4. Corrector — wave.AdvanceADER applies the imposed-flux side-channel while re-running the ADER bulk update, producing  $Q_{\text{new}}$ .



## F. Example TOML (production-ready EXAMPLE)

```
# safs/project_7.0_alternative/friction/EXAMPLE_spatial_friction_slip_weakening_safs.toml
[meta]
schema_version = 1
law             = "slip_weakening"

[material_constant_fallback]
lambda = 32.0e9; mu = 32.0e9; rho = 2670.0    # TPV205-style constant material

[mesh]
path = "safs/project_7.0_alternative/meshing/results/msh/safs_fault_box_nwcut_1000m_lcfar3000.msh"
order = 1

[stress]
kind = "constant_tensor"
sigma_xx_pa = 80.0e6; sigma_yy_pa = 80.0e6; sigma_zz_pa = 80.0e6
sigma_xy_pa = 20.0e6; sigma_yz_pa = 0.0; sigma_xz_pa = 0.0

[numerics]
ader_order = 2; mixed_flux = "none"; cfl = 0.5

[time]
tfinal = "12s"; dt_max = "0.1s"

[nucleation]
kind = "gradual overstress"

[nucleation.gradual overstress]
center_x_m = 480000.0; center_y_m = 3750000.0; center_z_m = -7000.0
radius_dip_m = 3000.0; radius_strike_m = 3000.0
delta_tau_dip_pa = 0.0; delta_tau_strike_pa = 25.0e6
T_nuc_s = 1.0

[friction.slip_weakening]
mu_s_default = 1.1; mu_d_default = 0.5; d_c_default = 0.5

[[friction.slip_weakening.spatial]]
kind = "barrier"; z_min_m = -20000.0; z_max_m = -15000.0

[[friction.slip_weakening.spatial]]
kind = "box"
x_min_m = 478000.0; x_max_m = 486000.0
y_min_m = 3748000.0; y_max_m = 3756000.0
z_min_m = -9000.0; z_max_m = -5000.0
mu_s = 1.05; mu_d = 0.5; d_c = 0.5
```

## G. Phase Z — invocation patterns

```
# Local 8-rank smoke (per feedback_local_mpi_cores)
conda activate mfem-dev
cd miniapps/seas
make seas_spatial_dyn_driver

mpirun -np 8 ./seas_spatial_dyn_driver \
  --config safs/project_7.0_alternative/friction/EXAMPLE_spatial_friction_slip_weakening_safs.toml \
  --mesh safs/project_7.0_alternative/meshing/results/msh/safs_fault_box_nwcut_1000m_zgraded.msh \
```

```

--no-sidecar-material \
--tfinal 0.5s \
--paraview --paraview-fault-hdf5 \
--paraview-fault-zfp-tol 1.0e-12 \
--paraview-max-snapshots 200 \
--print-derived \
--dry-run          # remove --dry-run to actually integrate
--output-dir /tmp/safs_dyn_smoke

# Post-run smoke verifier
conda activate pythonenv
python safs/project_7.0_alternative/spatial/code/scripts/verify_spatial_dyn_smoke_safs.py \
--fault-vtkhdf /tmp/safs_dyn_smoke/fault.vtkhdf \
--toml-config safs/project_7.0_alternative/friction/EXAMPLE_spatial_friction_slip_weakening_safs.toml

```

For Frontera production, jobs/safs/spatial\_dyn\_production\_normal\_24hr\_safs.sbatch adds `--tfinal 12s --checkpoint-every 10000` and the regime-adaptive cadence flags `--paraview-coseismic-dt 0.005 --paraview-nucleation-dt 0.001`.

## Conventions worth knowing

| Topic                      | Convention                                                                                                                                                                                                            |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Sign — $\sigma_n$          | $\sigma_n > 0$ is compression (geology). <code>min_sigma_n_pa</code> clamps from below to keep effective stress positive.                                                                                             |
| Sign — slip-rate           | $V_{\text{vec}} = (V_{\text{abs}} / \tau_{\text{abs}}) \cdot \tau_{\text{vec}}$ — parallel to traction, not antiparallel. Antiparallel makes positive feedback (debug v8).                                            |
| Sign — $\tau_{\text{pre}}$ | Parallel to initial velocity: $\tau_{\text{pre}} = \tau_{0\_scalar} \cdot V_i /  V_i $ .                                                                                                                              |
| Fault basis                | BP5 / FaultBasis (Tandem): <code>tangent1</code> = dip (downward), <code>tangent2</code> = strike.<br>D0FData:: <code>{tau1,V1,slip1,tau1_nuc}</code> are DIP components; *2 are STRIKE.                              |
| Dip direction              | (0, 0, +1) downward into earth.                                                                                                                                                                                       |
| Mesh free surface          | <code>mesh_zmax == 0</code> is invariant — never bake an aerial slab.<br><code>run_nwcut_meshing.py</code> enforces this via<br><code>--fault-top-clamp == --pad-top</code> (both default 100 m).                     |
| Mesh format                | Gmsh v2.2 ASCII only ( <code>mfem::Mesh::ReadGmshMesh</code> is a v2.2-only parser; v4 aborts with the misleading vertices indices are not unique).                                                                   |
| Friction solver            | Brent's method (not Newton). Newton fails on large $\psi/a$ because $F(V_{\text{lo}}=1e-30) > 0$ while the true solution sits below $1e-30$ (debug v1).                                                               |
| CFL                        | $dt = cfl \cdot h_{\text{min}} / cp$ with scalar $cp = \sqrt{(\lambda + 2\mu) / \rho}$ from <code>[material_constant_fallback]</code> .<br>Heterogeneous CFL is a follow-up.                                          |
| Output mode policy         | <code>MFEM_USE_HDF5=YES</code> builds default to <code>FaultOutputMode::Hdf5</code> (single fault.vtkhdf).<br><code>--paraview-fault-zfp-tol 1e-12</code> preserves slip-rate dynamic range; bulk/volume use $1e-3$ . |
| Checkpoint format          | Per-rank <code>cp_t&lt;step&gt;.h5</code> via<br><code>Write/ReadTpv104Checkpoint</code> with <code>driver_tag = "spatial_dyn"</code> . Restart MUST use the same <code>ibrun</code> rank count.                      |

| Topic           | Convention                                                           |
|-----------------|----------------------------------------------------------------------|
| MPI requirement | MFEM_USE_MPI = YES is mandatory (the driver has #error at line 789). |

## Gotchas (the non-obvious things)

1. **MaterialField::Mode::Coefficient** is rejected at startup. The heterogeneous WaveOperator(MaterialField, BoundaryConfig) ctor is the Phase H Stage 2 deferred work (dynamic/wave\_operator.hpp:215 — SetGondunovFluxPool still aborts). The driver hard-requires `--no-sidecar-material` or a velocity sidecar that resolves to `Mode::Constant` until Phase R lands. This is enforced at driver line 856.
2. **dummy\_T\_forced(=1e9) / dummy\_t0\_decay(=0)** are intentional sentinels. `InitializeFaultDOFs_Spatial` validates that those two Vectors have size `num_fault_total` (spatial\_setup.hpp:194–200). Under the LSW dispatch (not LSW\_ForcedRupture), `EvaluateADER_LSW` never consults them, so the driver supplies `T_forced = 1e9 s` (“never forced”) and `t0_decay = 0` to satisfy the size check without bloating the API.
3. Shared-fault DOFs see Phase-N perturbations as a step, not a ramp. The driver emits a rank-0 WARNING (line 1058–1071) when the shared-fault fraction is non-zero — the wave operator’s shared-face `EvaluateADER_LSW` reads `DOFData::tau{1,2}_nuc` AFTER the per-sub-step iterator has applied the full macrostep increment. This is a 1st-order time-accuracy degradation on those DOFs; tighten dt to mitigate.
4. **dof\_basis** is (9, N) column-major. The original (3\*N, 3) layout claim was wrong (R-001 fix in the plan). Use `&dof_basis(3, i)` and `&dof_basis(6, i)` to get contiguous 3-vector pointers for dip and strike without copying — `GaussianFactorFaceLocal` consumes them directly.
5. **FaultGeometry::NumZeroNormalFallbacks()** can be non-zero on SAFS. The curvilinear ALT6 fault produces DOFs at degenerate orientations; the fallback rotates `GaussianFactorFaceLocal`’s input incorrectly at those DOFs. `--print-derived` reports the count; ignore at your peril.
6. Native TPV\* / BP5 drivers are deliberately isolated from this work. Phase N removed `NucleationKind::StrengthReduction` and `::Overstress` from the spatial path AND deleted `ResolveForcedRupture / ResolveOverstress`, but the native TPV104 / TPV205 / BP5 drivers keep their `tpv104_nucleation.hpp` + `LSW_ForcedRupture` machinery verbatim. The Phase N audit gate (PLAN\_first\_safs\_run.md §11) greps the native driver files for `NucleationKind|GradualOverstress|spatial_nucleation` and requires zero hits — the regression contract is byte-exact.
7. **TPV205SubStepIterator** has two **AdvanceWithSubStepStates** overloads. The original (no-callback) overload is preserved and routes to the new overload with a no-op `[(real_t, real_t){}]`. Existing TPV205 tests link against the original overload unchanged. If a future code path uses the iterator’s `Advance` (time-averaged  $\bar{Q}$ ) overload, the callback hook must be added there too (Plan R-004 TODO).
8. **paraview\_enabled** is a HARD master gate. Even if individual per-collection modes (`paraview_volume = "hdf5"` etc.) are set, NO output is written unless `paraview_enabled = true` (or `--paraview` on CLI). The Parity Phase 2 default flipped `paraview_volume / paraview_bulk` from `"hdf5"` to `"off"` to enforce opt-in posture.
9. Output-dir overwrite guard. When restarting, if `--output-dir` canonicalises to the same path as the parent of `--restart`, the driver exits 3 without doing anything — pick a different `--output-dir` for the resumed run.
10. **atan2** ordering for  $\sigma_1$  azimuth. The Phase D `--print-derived` pre-flight computes azimuth (clockwise from north in EAST-NORTH) as `atan2(v1x, v1y)`, NOT `atan2(v1y, v1x)`. Plan R-004 documents this explicitly; getting the ordering wrong silently rotates the reported  $\sigma_1$  direction by 90°.

## Open questions / follow-ups (per plan §Appendix R-XXX)

- R-001 — `GaussianFactorFaceLocal` at zero-normal-fallback DOFs: currently a [derived] WARNING; needs `FaultGeometry` basis fix.
- R-002 — `--print-derived`  $\sigma_1$  azimuth from sidecar stress: per-DOF azimuth histogram not yet implemented (depth-dependent eigenframe).

- R-003 — Automated V1-checkpoint restart round-trip test from a Phase 4/5 spatial\_dyn checkpoint into the post-Phase-N driver.
  - R-004 — Callback hook in `Tpv205SubStepIterator::Advance` (the time-averaged  $\bar{Q}$  overload). Only the `Advance-WithSubStepStates` overload is wired today.
- 

## Pointers for further reading

- `safs/project_7.0_alternative/document/spatial_dynamic_rupture_plan.md` — parent plan
- `safs/project_7.0_alternative/document/spatial_friction_config_schema.md` — TOML schema
- `safs/project_7.0_alternative/debug_document/spatial_workflow_safs_runbook_2026-05-18.md` — user runbook
- `safs/project_7.0_alternative/debug_document/spatial_paraview_compaction_parity_plan_2026-05-18.md` — ParaView parity plan
- `miniapps/seas/CLAUDE.md` — numerical conventions + ParaView mode policy
- `miniapps/seas/ARCHITECTURE.md` — class hierarchy + data flow
- `miniapps/seas/dynamic/tpv104_nucleation.hpp` — reference for `smoothStep` helpers
- `miniapps/seas/dynamic/tpv205_substep_iterator.{hpp,cpp}` — reference iterator
- `miniapps/seas/dynamic/spatial_nucleation.{hpp,cpp}` — Phase N source of truth
- `miniapps/seas/dynamic/spatial_print_derived.{hpp,cpp}` — Phase D source of truth